

面向 UR10 批量逆运动学的 CUDA-V4-Final-K16-OPT4C 求解器

匿名稿（通讯作者信息在单独标题页）

（作者单位信息在单独标题页）

文章编号：（预留） DOI：（预留）

摘要：批量逆运动学（inverse kinematics, IK）在采样规划、轨迹优化和机器人学习中需要对大量目标位姿重复求解小规模非线性最小二乘问题。面向官方 UR10 模型，本文实现并评估 CUDA-V4-Final-K16-OPT4C 求解器：以解析雅可比和 Levenberg-Marquardt (LM) 迭代为核心，结合 Sobol-K16 多种子、关节限位 barrier、平滑性重排序、target-block seed-parallel 线程映射与 block 内融合候选选择。该实现使用官方 UR10 URDF 导出的同一运动学常量，末端坐标系固定为 tool0；每个 target 映射为一个 CUDA block，block 内并行处理 16 个 Sobol 种子并完成最佳候选选择，减少额外候选筛选 kernel 和主机端后处理。基于 2026-06-16 最新重跑结果，在 NVIDIA GeForce RTX 4060 Laptop GPU 上，CUDA OPT4C 在 $N = 100, 500, 1000, 5000$ 时分别达到 15.5k、16.3k、17.8k、18.5k targets/s，Strict 成功率稳定为 0.954–0.960，整体位置误差 $p95$ 为 4.34–4.56 mm，且无 NaN/Inf。与默认 cuRobo-Graph 的系统级对比表明， $N = 100$ 时 CUDA OPT4C 在吞吐和精度上同时占优； $N \geq 500$ 时默认 cuRobo-Graph 吞吐更高，但 Strict 成功率较低且失败尾部导致整体位置误差 $p95$ 达 75–116 mm。本文结论是：低批量和质量敏感场景中，定制 CUDA V4 求解器能提供更稳定的收敛质量；大批量吞吐场景中，cuRobo-Graph 仍具有更强调度和流水化优势。

关键词：逆运动学；CUDA；UR10；解析雅可比；Levenberg-Marquardt；cuRobo

中图分类号：TP391.4 **文献标志码：**A

收稿日期：（预留）；**修回日期：**（预留）；**网络优先出版日期：**（预留）。

CUDA-V4-Final-K16-OPT4C

analytical Jacobian; Levenberg-Marquardt; cuRobo

Solver for Batch Inverse Kinematics of UR10 Manipulators

（Author information on separate title page）

Abstract: This paper presents CUDA-V4-Final-K16-OPT4C, a native CUDA solver for batch inverse kinematics of the official UR10 model. The solver combines an analytical Jacobian, Levenberg-Marquardt iterations, Sobol-K16 multi-seed initialization, a joint-limit barrier, smoothness reranking, target-block seed-parallel mapping, and fused in-block candidate selection. Using the latest rerun on 2026-06-16, the solver reaches 15.5k–18.5k targets/s on an NVIDIA GeForce RTX 4060 Laptop GPU, with a stable Strict success rate of 0.954–0.960 and all-target position-error $p95$ of 4.34–4.56 mm. A system-level comparison against default cuRobo-Graph shows that CUDA OPT4C is faster and more accurate at $N = 100$, while cuRobo-Graph provides higher throughput for $N \geq 500$ but lower Strict success rate and larger failure-tail errors. The results characterize a practical boundary between a quality-stable custom CUDA IK solver and a highly optimized graph-based general GPU optimization pipeline.

Keywords: inverse kinematics; CUDA; UR10;

1 引言

机械臂批量逆运动学是采样式运动规划、轨迹优化、抓取候选评估和机器人学习中的基础计算模块。给定一批末端目标位姿，求解器需要在关节限位内寻找一组或多组关节角，使正运动学输出的平移和旋转误差同时满足阈值。单个 6 自由度机械臂 IK 问题规模很小，但批量场景需要对数百至数千个目标重复执行正运动学、雅可比构造、小矩阵线性求解和候选选择，因此固定调度开销、主机-设备同步和失败尾部都可能主导真实性能。

现有机器人软件通常采用两类路线。第一类是 CPU 数值 IK 或解析 IK 工具链，例如 KDL、TRAC-IK 和基于 UR 解析几何的求解器，其接口成熟但在大批量目标下吞吐有限。第二类是 GPU 优化流水线，例如 cuRobo，其通过 CUDA Graph、并行优化器和内部种子策略提升端到端吞吐，适合大批量规划和优化任务。然而，对固定 UR10 模型、固定 $K = 16$ 种子和固定评价阈值的场景，通用优化流水线并不总是等价于专门的 IK kernel；特别是在低批量和质量敏感实验中，候选生成、限位约束和筛选规则会显著影响 Strict 成功率和误差尾部。

本文围绕官方 UR10 模型实现 CUDA-V4-Final-

K16-OPT4C (下文简称 CUDA OPT4C)。该方法不是重新搜索超参数,而是将已冻结的 V4-Final-K16 算法移植到 GPU 原生实现,并针对线程映射和候选选择进行 OPT4C 优化。最终方法由解析雅可比、LM 迭代、Sobol-K16 多种子、关节限位 barrier、平滑性重排序、target-block seed-parallel mapping 和 block 内融合候选选择组成。

本文贡献如下:

1. 给出面向 UR10 官方 URDF/tool0 坐标系的一致 CUDA IK 实现,解析雅可比显式使用各关节世界轴 z_i 与关节位置 p_i ,避免数值差分雅可比带来的额外 FK 调用。
2. 提出 target-block seed-parallel 线程映射:每个 target 对应一个 CUDA block,block 内并行处理 $K = 16$ 个 Sobol 种子,并在同一 block 内完成候选筛选。
3. 在同一目标集、同一 UR10 模型和同一评价阈值下,重跑 CUDA OPT4C 与默认 cuRobo-Graph 的系统级对比,明确低批量质量优势与大批量吞吐边界。
4. 将全部论文数字绑定到最新 CSV 结果,避免使用旧版本 A/B 实验、旧图片或过期吞吐结论。

2 问题定义与评价协议

2.1 UR10 批量 IK

令 $q \in \mathbb{R}^6$ 表示 UR10 六个旋转关节角,关节顺序固定为

shoulder_pan_joint, shoulder_lift_joint, elbow_joint, wrist_1_joint, wrist_2_joint, wrist_3_joint. (1)

正运动学记为

$$T_{ee}(q) = \begin{bmatrix} R_{ee}(q) & p_{ee}(q) \\ 0 & 1 \end{bmatrix} \in SE(3), \quad (2)$$

其中末端 frame 固定为 URDF 中的 tool0。对目标位姿 T^* , 平移误差和旋转误差分别为

$$e_p(q) = p_{ee}(q) - p^*, \quad (3)$$

$$e_R(q) = \log(R^{*\top} R_{ee}(q))^\vee. \quad (4)$$

批量 IK 的目标是在 N 个目标上并行求解

$$\min_q \frac{1}{2} \|e_p(q)\|^2 + \frac{1}{2} \alpha_R \|e_R(q)\|^2 + \Phi_{\text{limit}}(q), \quad (5)$$

并输出每个 target 的最佳候选解、误差、迭代次数和限位指标。

2.2 统一评价指标

本文采用统一求解后多阈值评价,不为某个方法单独改变成功判定。主表报告:

- Strict 成功率: 同时满足严格平移和旋转阈值的 target 比例;
- all-target 位置误差 $p95$: 对全部 target 统计位置误差 95 分位数,失败样本也计入;
- near-limit 比例: 最终解接近关节限位 margin 的比例;
- raw throughput: $N/\bar{t}_{\text{gpu}}$;
- valid throughput: Strict 成功 target 的吞吐。

all-target $p95$ 比仅统计成功样本的误差更能体现失败尾部风险,因此本文在比较 cuRobo 时同时关注 Strict 成功率和 all-target $p95$ 。

3 CUDA-V4-Final-K16-OPT4C 方法

3.1 官方 UR10 模型与坐标系

实验使用 'standard_robot_cuda_ik/urdf/ur10_official.urdf' 作为统一模型来源,并将模型常量生成到 CUDA 头文件中。CUDA 和对比脚本均使用官方 UR10 而非 UR10e,末端坐标系固定为 tool0。模型一致性审计表明, joint order、base frame 和 tool frame 必须在 CUDA、URDF/KDL 和 cuRobo 配置之间显式对齐;否则 70-110 mm 量级的固定误差可能由 tool0/flange/ee_link 偏移或非官方模型混用造成。

3.2 带关节 frame 的 FK

CUDA FK 函数不仅输出 T_{ee} , 还输出每个关节在世界坐标下的位置 p_i 和转轴 z_i 。第 i 个关节轴的计算为

$$z_i = R_i a_i, \quad (6)$$

其中 R_i 是当前关节前的世界旋转矩阵, a_i 是该关节在局部坐标系中的轴向量。该公式避免将旋转矩阵列向量误写为叉乘形式。输出的 p_i 与 z_i 直接用于解析雅可比。

3.3 解析雅可比

对旋转关节，几何雅可比为

$$J_{v,i} = z_i \times (p_{ee} - p_i), \quad (7)$$

$$J_{\omega,i} = z_i. \quad (8)$$

因此雅可比矩阵 $J \in \mathbb{R}^{6 \times 6}$ 由三维线速度项和三维角速度项拼接得到。与有限差分雅可比相比，解析雅可比每轮不再需要对每个关节做正负扰动 FK，降低了单 seed 迭代成本，并消除了差分步长对数值稳定性的影响。

3.4 LM 更新与 Limit Barrier

每个 seed 执行 LM 迭代。设加权误差为 e ，雅可比为 J ，阻尼因子为 λ ，则求解

$$(J^T J + \lambda I) \Delta q = -(J^T e + w_{\text{limit}} \nabla \Phi_{\text{limit}}). \quad (9)$$

本实现固定 $w_{\text{limit}} = 0.03$ ，限位 margin 为 0.087 rad ，当前 benchmark 使用解析限位梯度。与标准带 rejection 的 LM 不同，V4 原型中的更新逻辑为： q_{trial} 总是接受， λ 仅根据新旧 loss 的相对变化自适应缩放。CUDA 版本逐行复现该逻辑，不额外加入 line search、trust region 或 rejection 分支。

3.5 OPT4C 线程映射与候选选择

OPT4C 将一个 target 映射为一个 CUDA block，block 内处理 $K = 16$ 个 Sobol 种子。每个 seed 维护独立的 q 、误差、迭代次数和限位状态，所有候选在 block 内完成筛选。静态 batch IK 的选择规则为

$$\text{success rank} \rightarrow \text{near-limit} \rightarrow \text{pose cost}. \quad (10)$$

轨迹场景在候选级后处理中额外加入 smoothness rerank。本文最新表格只报告 static batch IK 与默认 cuRobo-Graph 对比；轨迹重排序属于同一 V4 算法族，但不是本轮 PDF 的吞吐主证据。

4 实验设置

4.1 硬件与软件

最新结果由 ‘standard_robot_cuda_ik/PROJECT.md’ 与 ‘data/results/latest/*.csv’ 记录。测试环境见表 1。

表 1: 实验环境

项目	当前值
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
Driver	610.43.02
CUDA Toolkit	13.3 / nvcc V13.3.33
主 runner	standard_robot_cuda_v4_runner
CUDA variant	opt4c_block_target
Precision	fp64
Limit gradient	analytic
K	16
Warmup / repeat	10 / 30

4.2 数据与对比口径

CUDA 输入包括四个批量规模 $N = 100, 500, 1000, 5000$ 的目标位姿 raw 文件和对应 Sobol-K16 seed raw 文件。cuRobo 对比使用默认 cuRobo-Graph，collision disabled，zero external seed，并记录 cuRobo 内部 optimizer、CUDA Graph 和 seeding 与 V4 Sobol-K16 不完全等价。因而本文不将该对比表述为完全等价算法对比，而是表述为统一目标集、统一 UR10 模型和统一评价协议下的系统级对比。

5 实验结果

5.1 CUDA OPT4C 静态批量 IK

表 2 给出最新 CUDA OPT4C 静态 batch IK 结果。四个批量规模全部无 NaN/Inf，monotonic check 通过。Strict 成功率在 $0.954\text{--}0.960$ 之间，all-target 位置误差 $p95$ 在 $4.34\text{--}4.56 \text{ mm}$ 之间，near-limit 比例低于 1.0% 。

从吞吐看，CUDA OPT4C 随批量增大从 15.5k targets/s 提升到 18.5k targets/s ，说明 kernel 固定开销在小批量下仍然可见；从质量看， $N = 500, 1000, 5000$ 的 Strict 成功率和 $p95$ 基本稳定，说明目标集扩展没有破坏算法质量。

5.2 与默认 cuRobo-Graph 的系统级对比

表 3 给出 CUDA OPT4C 与默认 cuRobo-Graph 的对比。 $N = 100$ 时，CUDA throughput 为 $15539.2 \text{ targets/s}$ ，cuRobo 为 $10508.7 \text{ targets/s}$ ，吞吐比为 1.479 ；同时 CUDA Strict SR 为 0.960 ，高于 cuRobo 的 0.870 ，all-target $p95$ 也显著更小。该结果说明在低批量场景

表 2: CUDA OPT4C 最新静态 batch IK 结果。来源: data/results/latest/cuda_opt4c_static.csv

N	GPU mean/ms	Throughput/s	Valid throughput/s	Strict SR	Pos $p95$ /mm	Rot $p95$ /deg	Near-limit	Iter mean
100	6.435	15539.2	14917.7	0.960	4.385	0.573	0.010	21
500	30.592	16344.0	15592.2	0.954	4.338	0.568	0.004	20
1000	56.123	17817.9	16998.3	0.954	4.563	0.530	0.007	19
5000	270.411	18490.4	17639.8	0.954	4.563	0.530	0.007	19

中, 专用 CUDA kernel 的固定开销较低, 且 Sobol-K16 与 V4 候选选择带来更稳定的质量。

$N \geq 500$ 时, 默认 cuRobo-Graph 吞吐快速提升, 显著高于 CUDA OPT4C; 但 cuRobo Strict 成功率保持在 0.836–0.844, all-target $p95$ 为 75–116 mm, 说明失败样本尾部较大。该现象不应简单解释为 cuRobo 算法本身劣于 CUDA V4, 因为默认 cuRobo-Graph 使用内部优化器、图捕获和 seed/config 策略, 而本文 CUDA 使用固定 Sobol-K16 和 V4 selection。更准确的结论是: 在当前统一目标集和评价协议下, 默认 cuRobo-Graph 体现了大批量吞吐优势, 而 CUDA OPT4C 体现了质量稳定性和小批量端到端优势。

5.3 质量门控与论文可用性

最新运行状态显示: build pass、FK check pass、CUDA static all-N pass、 $N=100$ quality gate pass、CUDA NaN/Inf gate pass、cuRobo-Graph pass。特别地, $N = 100$ 时 Strict SR=0.960、pos_p95_all=4.3845 mm, 满足 Strict SR ≥ 0.90 和 pos_p95_all ≤ 10 mm 的验收阈值。四个 CUDA 规模均没有 NaN/Inf, 因此 CUDA OPT4C 最新数据可以作为论文中的主表数据。

6 讨论

6.1 CUDA OPT4C 的适用边界

CUDA OPT4C 的优势来自三个方面。第一, 算法结构固定, UR10 官方模型、tool0 frame、Sobol-K16 和 V4 selection 都在编译和运行层面被显式控制。第二, 解析雅可比避免了有限差分 FK 的重复调用, 使单 seed 迭代更轻量。第三, target-block seed-parallel mapping 将候选生成和候选选择融合在一个 block 内, 减少全局内存读写回和额外筛选开销。

另一方面, CUDA OPT4C 并不试图替代通用 GPU 优化框架。cuRobo-Graph 在大批量下通过图捕获、内部并行优化和成熟流水线显著提升 throughput。表 3 显示, 当 N 从 100 增加到 5000, cuRobo throughput 从 10.5k/s 提升到 137.1k/s, 而 CUDA OPT4C 保持

在 15.5k–18.5k/s。这说明当前 OPT4C kernel 的吞吐上限仍受每 target 固定迭代成本、FP64 小矩阵计算和 block 内串行控制逻辑限制。

6.2 cuRobo 误差尾部的解释

当前 cuRobo all-target $p95$ 为 74–116 mm, 远大于 CUDA OPT4C 的 4.34–4.56 mm。该差异可能来自默认 cuRobo 配置的内部 seed、优化器终止条件、目标 frame 处理、失败样本统计方式和候选选择策略。前期模型一致性审计指出, UR10/UR10e 混用、非官方 URDF、tool0/flange/ee_link 不一致或 joint order 错误都可能造成 70–110 mm 量级误差。因此, 论文中应谨慎表述: 当前结果证明了在本项目最新统一输入和评价脚本下的系统级边界, 但若要给出更强的 cuRobo 质量结论, 还需要进一步重跑 quality-tuned cuRobo 或更严格的 FK cross-check。

6.3 真机实验限制

本文使用官方 UR10 通用模型, 没有加载真实机器人 factory calibration。UR 工业机械臂的真实 DH/kine-matics calibration 会影响毫米级精度。因此, 在真机实验前必须通过 ‘ur_calibration’ 提取真实 UR10 calibration, 并由同一校准模型生成 CUDA、cuRobo 和 ROS2/MoveIt 共享的 URDF/YAML 配置。未完成该步骤前, 仿真 benchmark 结论不能直接等同于真机 TCP 精度结论。

7 结论

本文基于最新 OPT4C 结果重写并验证了 UR10 批量 IK 的 CUDA 实验结论。CUDA-V4-Final-K16-OPT4C 在 RTX 4060 Laptop GPU 上实现了 15.5k–18.5k targets/s 的静态 batch IK 吞吐, Strict 成功率稳定在 0.954–0.960, all-target 位置误差 $p95$ 为 4.34–4.56 mm, 且无 NaN/Inf。与默认 cuRobo-Graph 的系统级对比显示, CUDA OPT4C 在 $N = 100$ 小批量下同时具备吞吐和质量优势; $N \geq 500$ 时 cuRobo-Graph 吞

表 3: CUDA OPT4C 与默认 cuRobo-Graph 系统级对比。来源: data/results/latest/cuda_vs_curobo_summary.csv

N	CUDA thr./s	CUDA SR	CUDA $p95/mm$	cuRobo thr./s	cuRobo SR	cuRobo $p95/mm$	Thr. ratio
100	15539.2	0.960	4.385	10508.7	0.870	74.324	1.479
500	16344.0	0.954	4.338	41815.6	0.836	115.637	0.391
1000	17817.9	0.954	4.563	64928.2	0.840	98.920	0.274
5000	18490.4	0.954	4.563	137148.3	0.844	75.047	0.135

吐更强,但当前默认配置下质量指标较弱。该结果为论文提供了清晰边界:本文的核心贡献不是宣称在所有批量规模全面超过 cuRobo,而是在固定 UR10、固定 Sobol-K16 和严格评价协议下,给出质量稳定、可审计、低批量延迟有优势的 CUDA IK 实现。

后续工作包括三部分:一是对 cuRobo 做 quality-tuned 配置重跑,区分默认吞吐模式与质量优先模式;二是基于 Nsight Compute 进一步定位 FP64、小矩阵解算、register pressure 和 branch divergence 的瓶颈;三是在真实 UR10 上引入 factory calibration,验证仿真误差与真机 TCP 误差之间的差异。

参考文献

- [1] Siciliano B, Sciavicco L, Villani L, et al. Robotics: Modelling, Planning and Control. Springer, 2010.
- [2] Buss S R. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods. IEEE Journal of Robotics and Automation, 2004.
- [3] Nakamura Y, Hanafusa H. Inverse kinematic solutions with singularity robustness for robot manipulator control. Journal of Dynamic Systems, Measurement, and Control, 1986, 108(3): 163-171.
- [4] Universal Robots. Universal Robots ROS2 Description. https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [5] Orocos Project. Kinematics and Dynamics Library. <https://www.orocos.org/kdl.html>.
- [6] Sundaralingam B, et al. cuRobo: Parallelized collision-free robot motion generation. NVIDIA, 2023.
- [7] NVIDIA. CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [8] NVIDIA. CUDA C++ Best Practices Guide. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.
- [9] Levenberg K. A method for the solution of certain non-linear problems in least squares. Quarterly of Applied Mathematics, 1944, 2: 164-168.
- [10] Marquardt D W. An algorithm for least-squares estimation of nonlinear parameters. Journal of the Society for Industrial and Applied Mathematics, 1963, 11(2): 431-441.
- [11] Sobol I M. On the distribution of points in a cube and the approximate evaluation of integrals. USSR Computational Mathematics and Mathematical Physics, 1967, 7(4): 86-112.
- [12] LaValle S M. Planning Algorithms. Cambridge University Press, 2006.
- [13] Beeson P, Ames B. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. IEEE-RAS International Conference on Humanoid Robots, 2015.
- [14] Chitta S, Sucan I, Cousins S. MoveIt! IEEE Robotics Automation Magazine, 2012, 19(1): 18-19.
- [15] NVIDIA. Nsight Compute User Guide. <https://docs.nvidia.com/nsight-compute/>.
- [16] NVIDIA. Nsight Systems User Guide. <https://docs.nvidia.com/nsight-systems/>.